MADRIGAL-PROTOCOL-LAB

КЕЙС ЗАКАЗЧИКА

VolgaBlob

Лаборатория протоколов

Восстановление недокументированного бинарного протокола поверх TCP

UMIRHack 2026 · кейс #02 · Мадригал · команда HTTP 418

Python 3.12 · PySide6 · dpkt · pytest · Docker

Нас двое



Сабадаш Павел

Архитектура · захват · интерфейс

@Pasha1778



Предков Никита

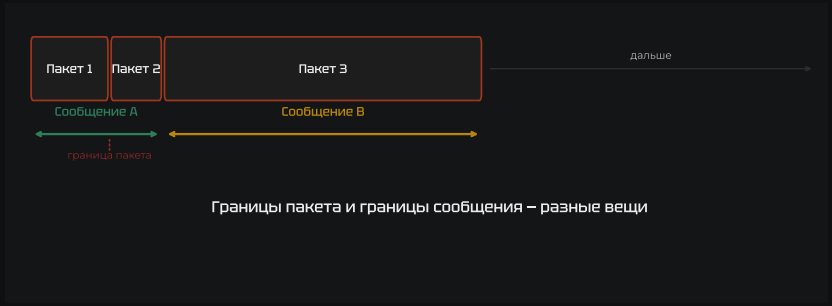
Правила · тесты · ревью

@petruan

Постановка задачи

TCP разрезает сообщения посередине

Границы пакета и границы сообщения — разные вещи



Дано и требуется

ДАНО

Захваты `pcap` / `pcapng` и журнал действий устройства

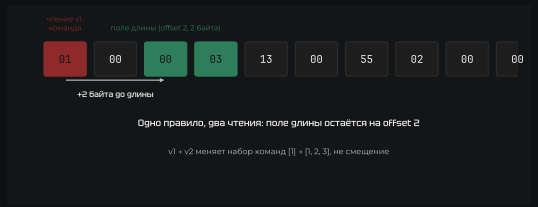
Спецификации протокола нет

ТРЕБУЕТСЯ

Проверяемая интерпретация структуры сообщений

Видно, где чтение верно, а где перестаёт работать

Смещение поля длины оказалось на четыре байта позже, чем мы считали: одно правило, два чтения, поле длины остаётся на offset 2.



Три модуля связаны форматами, а не вызовами

capture

pcap/pcapng → сессии reassembly, provenance

JSON →

protocol + hypothesis

правила, фрейминг контрпримеры, версии

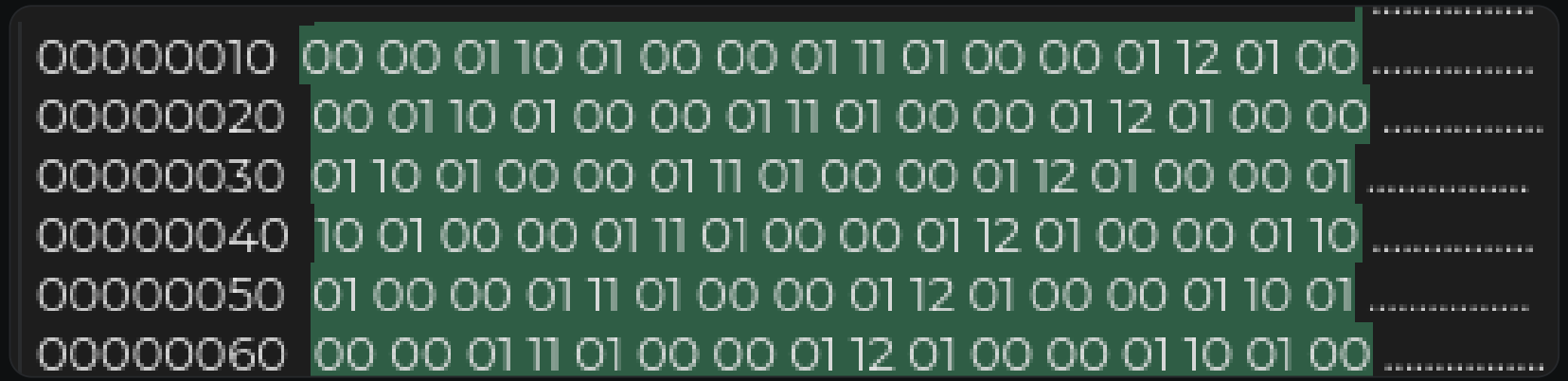
JSON →

ui

окно PySide6 исследование и просмотр

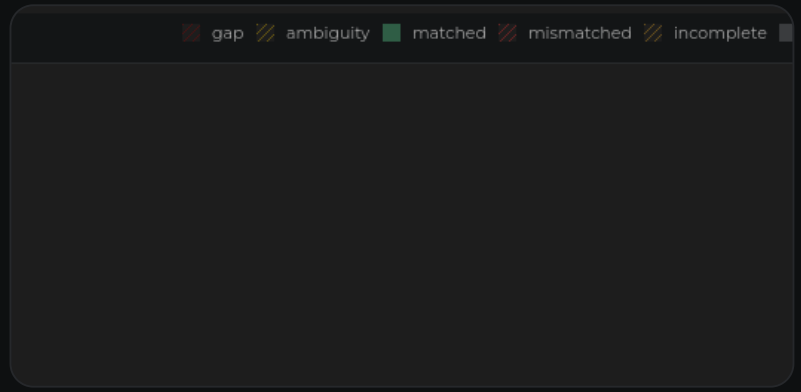
Контракты между модулями зафиксированы схемами JSON; порядок вызовов не важен

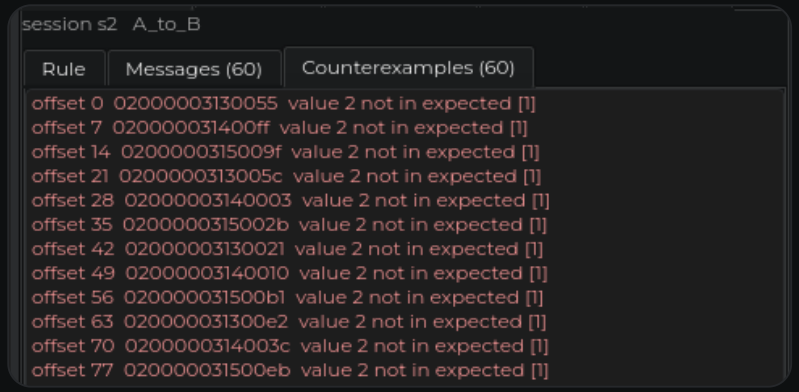
От интерпретации можно вернуться к исходным данным




```
packet 3 · seq 1001 · ts 1700000000.004 · сессия s1
```

Правило проверяется на всём корпусе сразу





4 независимые сессии в корпусе + журнал действий устройства

Контрпример важнее метрики

```
offset 0  02000003130055  value 2 not in expected [1]
offset 7  020000031400ff  value 2 not in expected [1]
offset 14  0200000315009f  value 2 not in expected [1]
offset 21  0200000313005c  value 2 not in expected [1]
offset 28  02000003140003  value 2 not in expected [1]
offset 35  0200000315002b  value 2 not in expected [1]
offset 42  02000003130021  value 2 not in expected [1]
offset 49  02000003140010  value 2 not in expected [1]
offset 56  020000031500b1  value 2 not in expected [1]
offset 63  020000031300e2  value 2 not in expected [1]
offset 70  0200000314003c  value 2 not in expected [1]
offset 77  020000031500eb  value 2 not in expected [1]
offset 84  02000003130047  value 2 not in expected [1]
offset 91  0200000314005b  value 2 not in expected [1]
```

Значение `2` повторяется в независимых сессиях корпуса и связано с действием «изменить настройку» в журнале

`02 00 00 03 13 00 55`

Команда `2` вне ожидаемого набора `[1]`

ОДИН БАЙТ — 80 КОНТРПРИМЕРОВ

80 несоответствий v1 — на offset 0 в сессиях `s2` и `s3`.

Что получилось: 60 из 140 → 140 из 140

На корпусе уточнения. Отдельно проверено на реальном датасете: 45 + 45, precision 1.00



Было 60 matched из 140 и 80 контрпримеров. Стало 140 matched из 140 и ноль контрпримеров

Осколки IP собираются в одно сообщение

IPv4 и IPv6 по своим правилам. Сессия знает, из каких пакетов собрана: статус reassembled_from

IPv4 группирует по `id` и `protocol`, offset в единицах по 8 байт; неполный фрагмент не собирается.

IPv6 группирует по `id`; перекрытие не допускается — датаграмма отбрасывается.



IPv4 id + protocol, offset ×8

IPv6 id, перекрытие отклонено

12 тестов фрагментации

Дырка — это incomplete, а не ноль: байты не угадываются, пока не покрыт каждый байт до конца и не увиден последний фрагмент

Набор организаторов разобран целиком

90

45 запросов + 45 ответов. Все matched.

12 с на 95 МиБ — пик памяти 350 МиБ, 9% бюджета 4 ГиБ



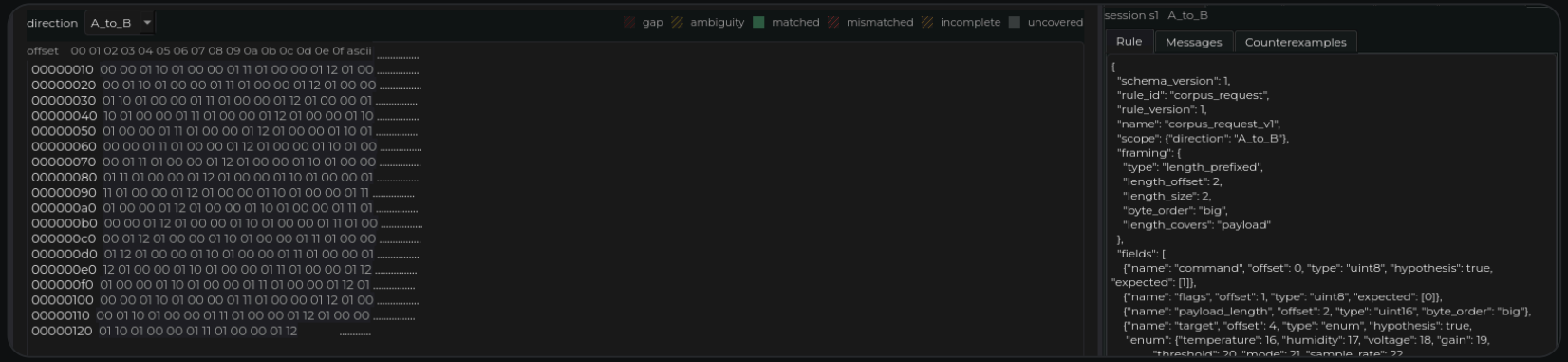
12 с общее время

350 МиБ пик памяти

250k пакетов

Данные не использовались при уточнении правила; условия замера: 95.66 МиБ, 250 000 пакетов, 1000 сессий, 4 vCPU / 16 ГиБ

Демонстрация: живое окно и запись прогона



Сессии, байты, интерпретация, происхождение и диагностика в одном окне. Резерв — запись полного прогона, 2:22, без звука; играется, если живое окно не поднимется

Границы и итог

ЧТО МЫ НЕ УТВЕРЖДАЕМ

Смысл байта флагов не установлен: в корпусе он постоянен

Поле значения остаётся гипотезой, а не фактом

ЧТО ОСТАЛОСЬ ЗА РАМКАМИ

Альтернативные чтения берутся из заданного набора

Интерпретация верна для проверенных захватов и молчит вне них

Инструмент превращает байты в проверяемое знание

Репозиторий

Релиз v1.0.0

@Pasha1778

@petruan

github.com/Pavel1778/madrigal-protocol-lab

VolgaBlob

кейс #02 · UMIRHack 2026